Learning Inverse Observables from Precomputed Koopman Operators

EECE 7345 Final Project

Alexander Estornell
Department of Computer Science
Northeastern University
Boston, United States
estornell.a@northeastern.edu

Alenna Spiro
Department of Computer Science
Northeastern University
Boston, United States
spiro.a@northeastern.edu

***Abstract*—A fundamental challenge in the estimation of temporal dynamical systems is finding linear representations for non-linear dynamics. Solutions to finding a linear operator which either approximates or fully captures the dynamics of a non-linear system fall into two categories: learning-based and programmatic. Learning-based approaches are efficient, however require a priori knowledge of the dimensionality of the linear estimator. Programmatic methods involve the use of computationally expensive semi-definite programs (SDP), however do not need a priori knowledge of its dimensionality. To address the limitations of both methods, we propose a hybrid framework that uses programmatic SDP methods to find a good representation of the linear operator, then uses a learning-based approach to approximate the coordinate transform used to and from the space our linear operator operates on. This approach ensures efficient learning of the operator and explicit coordinate transforms while avoiding the need for erroneous pre-specification.**

I. Introduction

In the context of dynamical systems, making predictions for unknown, non-linear systems is a foundational problem. The ability to predict the state of a non-linear system is used in a variety of engineering fields, ranging from path-planning for robotic systems to predicting the aircraft dynamics of rockets. A popular and computationally tractable [9] method for modeling these systems is through the use of a Koopman operator. First described by Bernard Koopman [4] in 1931, the Koopman operator is an infinite dimensional linear propagator that acts in the space of observables, which encodes the dynamics of a non-linear system. If the conversion from the state space to the observable space is known, the system can be propagated forward in time taking advantage of the properties of linearity. Linear dynamics are traditionally simple to propagate forward in time by virtue of their representation in matrix form for computational ease and the existence of closed-form solutions. Non-linear dynamics do not necessarily have a closed-form solutions, may have unstable characteristics. Thus efficient and accurate linearizations of non-linear systems are highly desirable.

Approaches to linearizing dynamical systems can be broadly cast into two categories: learning-based and programmatic. Learning-based approaches, such as [5], capture the transformation from the state space to the Hilbert space of observables and its inverse in the form of an autoencoder. Autoencoders are dual-network systems that encode given inputs into some latent, often lower dimensional, space that must then be recovered by the decoder as output. The resulting system learns an accurate estimated embedding of the data. Within the context of linearlizing non-linear dynamical systems, these two networks learn the Koopman eigenfunctions which translate between the state space and the space of observables where the Koopman exists. In Lusch's [5] method, an additional auxiliary network is used to estimate the Koopman itself between the two original networks, composing of a three network system that maps from a previous state to the next state. In contrast to learning approaches, programmatic approaches such as Sznaier's [8] reframe this as a convex optimization problem. Sznaier uses a rank-constrained Semi-Definite Program (SDP) to solve for the embeddings and koopman model directly, then uses a second SDP to reconstruct states.

However, these approaches are also not without their limitations. Learning-based approaches often require a priori knowledge of the embedded space's dimensionality, in order to correctly structure the neural networks before training. Choosing a latent space which is too large may require significantly higher amounts of data to train well, while choosing a latent space which is too small may not capture the dynamics sufficiently and render the learned Koopman useless. Semi-definite programs also often require a significant amount of computation to solve, and in Sznaier's case requires an entire additional SDP to produce resulting next-states. To bridge the gap between these two approaches and to leverage their unique advantages while alleviating their drawbacks, we propose a hybrid approach to finding linear approximations for nonlinear dynamics. To do this we develop a two-stage optimization scheme which first uses a modified semi-definite program to find an approximate koopman operator whose dimension is optimized for, then applies this knowledge to train an autoencoder to find the observable mappings. We attempt two different implementations to compare, one which only uses

the SDP-derived koopman, and the other which uses the SDP-derived koopman as a hot-start for the koopman auxillary network defined in Lusch [5]. Our results show that both models vastly outperform the baselines on 1-step predictions, with the hot-start method achieving the best performance.

II. Background

A. Dynamical systems

For the purpose of this work, we focus on discrete dynamical systems of the form:

$$x_{k+1} = f(x_k) \tag{1}$$

Where $x_k \in \mathbb{R}^n$ is a state, and f propagates the state forward in time. It is common for f to be unknown and non-linear. Systems that are, in fact, linear take the form:

$$x_{k+1} = Ax_k \tag{2}$$

This allows for easy propagation of a system with the closed form solution

$$x_{k+m} = A^m x_k \tag{3}$$

Therefore, it is appealing to work with linear systems.

B. Koopman Operator Theory

For any deterministic discrete dynamical system, there exists a [1] Koompman operator, $\mathcal{K}$, That is able to linearly propagate observable functions

$$\mathcal{K}\psi(x_k) = \psi(x_{k+1}) \tag{4}$$

where ψ is some m-dimensional measurement of the state, x_k. Since $\mathcal{K}$ operates on the Hilbert space of observable functions, it is infinite dimensional. Much work [7, 9] has gone into finite approximations of K and capturing its associated observable space.

Many of these methods involve finding eigenfunctions of the Koopman operator. Eigenfunctions are guaranteed to span a Koopman-invariant subspace [2], which allows you to perfectly approximate the the system linearly with a finite representation.

C. Approximating the Koopman through an SDP

Equations (10)-(12) in [8] describe a rank minimization SDP that when solved produce observables of the input set and a Koopman operator. This is done using a kernel trick and a Loewner embedding, utilizing the fact that if a Loewner matrix is rank deficient, then the existence of a function that maps states to there observables is guaranteed [3]. This method benefits from the fact that the dimension and structure of the Koopman is found during optimization, but finding these embeddings is expensive and the quality of the observable mappings depend heavily on their input's closeness to the training set.

D. Finding linear embeddings through deep learning

Due to the computational efficiency of neural networks, [5] is able to use an autoencoder structure to learn the Koopman operator and a representation of its eigenfunctions. The objective is to have the encoder, $\varphi(\cdot)$, map states, x, to coordinates, $y = \varphi(x)$, that span the eigenfunctions of their learned Koopman operator. This leverages the Koopman-invariant subspace property mentioned before to maximize the quality of the finite dimensional Koopman approximation. The networks ability to learn the Koopman operator and these intrinsic coordinates are due to three special loss functions that we outline in the following section. This method also extends to continuous spectra systems but due to time considerations, we do not consider this extension.

III. Methodology

At a high level, our approach has two phases: first determine the dimensionality and finite-dimensional approximation of the Koopman Operator, then use these components to create an Autoencoder network to learn the mappings to and from the embedded space. We explore two approaches, one using the original Koopman Operator and training only the mappings, and one using the given operator as a hot start for an auxillary network which is trained alongside the mappings to improve the performance of the system.

A. Koopman Extraction

We solve the convex SDP outlined in Algorithm 1 of [8] to determine an approximate Koopman operator and dimension of its associated Hilbert space. This takes in a set of states in either a single or multiple trajectories to determine the dimension of the embedded observable space and estimated Koopman operator.

B. Learned Dynamics

Once the dimensionality and estimate of the Koopman has been found, we then turn to the learning phase of our pipeline. Our system is comprised of three neural networks: an encoder, auxillary network, and decoder arranged in a similar structure to Lusch [5], shown in figure 1. To train the networks, we use the following loss function, which is composed of several sub-objectives:

a) Reconstruction Loss: We seek to train the network to learn the intrinsic coordinate transforms $y = \varphi(\boldsymbol{x})$ and $\boldsymbol{x} = \varphi^{-1}(\boldsymbol{x})$ required to translate in and out of the space of observables regardless of the forward dynamics of the system. Our autoencoder's encoder component learns the embedding into the observable space $y = \varphi(\boldsymbol{x})$, while the decoder learns the inverse into the state space $\boldsymbol{x} = \varphi^{-1}(\boldsymbol{x})$. Reconstruction accuracy of the system is defined by the following loss:

$$RLoss(\boldsymbol{x}) = ||\boldsymbol{x} - \varphi^{-1}(\varphi(\boldsymbol{x}))||_2^2$$

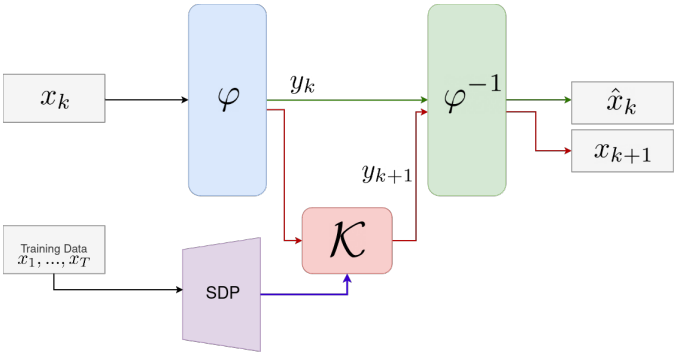


Fig. 1: Our model and training scheme. Green lines refer to the data path when calculating the Reconstruction loss. Red lines refer to the data path when calculating the Linearity and Forward losses. When the auxillary network is being used for training, $\mathcal{K}$ refers to the network and the purple lines are removed.

b) Linearity Loss: In addition to learning the observable mappings, we intend to preserve the linear dynamics of the system afforded by the Koopman Operator, whether given or learned. A single-step forward prediction can be described as $\boldsymbol{x}_{k+1} = \varphi^{-1}(K\varphi(\boldsymbol{x}_k))$. More generally, we can enforce linear predictions over m time steps with the following loss:

$$LLoss(\boldsymbol{x}, m, K) = ||\varphi(\boldsymbol{x}_{k+m}) - K^m\varphi(\boldsymbol{x}_k)||_2^2$$

c) Forward Loss: The final property the network must learn is that the intrinsic coordinates must enable future state prediction. This loss primarily identifies the linear dynamics in the matrix K, which shapes the behavior of the auxillary network when active with the hot-start method. When the auxillary network is replaced entirely by the given Koopman Operator derived from the SDP, this function enforces a relationship between the two independent Reconstruction and Linearity losses. When not using the auxillary network, any innacuracies that exist in the given Koopman may be more substantial in this loss function, however it is still needed for accurate predictions.

$$FLoss(\boldsymbol{x}, m, K) = ||\boldsymbol{x}_{k+m} - \varphi^{-1}(K^m\varphi(\boldsymbol{x}_k))||_2^2$$

We make the further modification of applying exponential decay to the aforementioned loss weights λ_2, and λ_3 in order to prioritize correct prediction of fewer steps. The full loss function is then:

$$Loss = \lambda_1 * RLoss_i + \sum_{i=0}^{m} \lambda_2^i * LLoss_i + \lambda_3^i * FLoss_i$$

Where $\lambda_{1,2,3} \in [0, 1]$ and $LLoss_i$, $RLoss_i$ refer to the linearity loss and reconstruction loss at the ith prediction, respectively.

IV. Experiments

A. Experimental Design

a) Environments: We compare our results on three unique dynamical systems: A fluid flow box, a fluid flow with an attractor, and a pendulum. Each of these is formulated as an ode system and data is generated using Lusch's [5] code.

The fluid flow with an attractor system considers the nonlinear fluid flow past a stationary cylinder at Reynolds number 100, which is characterized by vortex shedding. This environment has been a benchmark in fluid dynamics and has been extensively studied [6]. It has been shown that the dynamics evolve when a low-dimensional attractor is present, producing the following model:

$$\dot{x}_1 = \mu x_1 - \omega x_2 + A_1 x_3, \tag{5}$$
$$\dot{x}_2 = \omega x_1 + \mu x_2 + A_2 x_3, \tag{6}$$
$$\dot{x}_3 = -\lambda\left(x_3 - x_1^2 - x_2^2\right). \tag{7}$$

The next system considered is a nonlinear pendulum. This system exhibits a continuous eigenvalue spectrum with increasing energy, which produces the following model:

$$\ddot{x} = -\sin(x) \Rightarrow \begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -\sin(x_1) \end{cases} \tag{8}$$

The final system is a fluid flow box, which is similar to the fluid flow with attractor, but without the cylinder present.

B. Experimental Setup

Our experiments consist of comparisons between four systems:

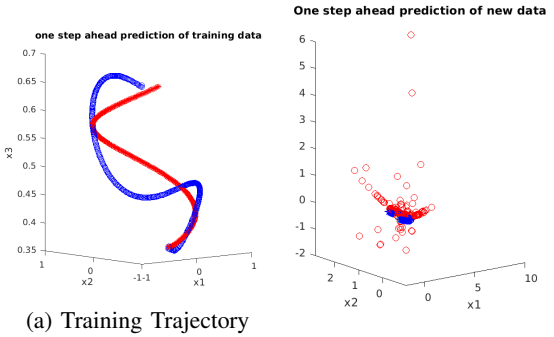
- Sznaier's original double-SDP implementation (Sznaier),
- A modern recreation of Lusch's three-network system (Lusch Net), NOTE: due to version incompatibility with Lusch's provided code, instead we used our method's network architecture without a hot-start.
- Our two-network system using the Koopman Operator given from Sznaier's first SDP formulation (Two Net Static Koopman),
- Our three-network system which uses the Koopman Operator as a hot-start (Three Net Hot Start).

a) Evaluation Metrics: In order to assess the efficacy of our systems, we have generated qualitative figures showing comparisons between the ground-truth trajectories and the 1-step predictions from each system. We also assess the accuracy of these predictions across the train, validation, and test sets.

C. Experimental Results

a) Double-SDP Implementation: First we tested Sznaier's implementation. Due to time constraints, we were unable to reconfigure the original codebase to allow for training on multiple trjectories. Due to this, this implementation learns using one trajectory rather than many. For the FluidFlowOnAttractor experiment 2, the system performed poorly on both the

training and test set. The remainder of the experiments for the Pendulum 3 and the Fluid Flow Box 4 recreate the training trajectories well, but again fail to recreate the test trajectories.

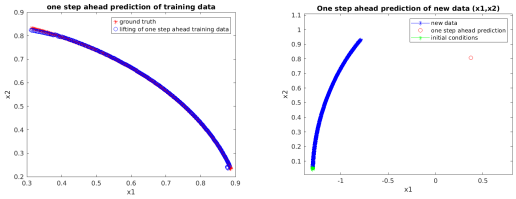


(a) Training Trajectory (b) Test Trajectory

Fig. 2: **Fluid Flow On Attractor**, Ground-Truth trajectories show blue. Predicted trajectories show red. On the test trajectory, the predictions were very poor, which has expanded the graph's size tenfold so that the original trajectory is no longer clearly visible.

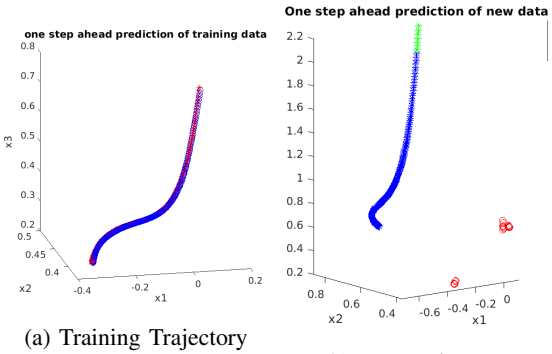
Sznaier's implementation of the second SDP to recover the predicted states took 9 minutes to produce per experiment. Each experiment considered 400 datapoints from a single trajectory and was unable to produce any informative results on test trajectories, whereas using the neural network implementations, we were able to train using 500 similar trajectories in 1/3 of the time to recover accurate predictions.

b) Direct Comparison of Remaining Results: The remaining results on the Lusch Net system, Three-Network Hot Start system, and Two-Network Static Koopman system are all directly comparable. We ran all experiments for 500 epochs with a learning rate of $1e^{-3}$ on an Adam optimizer. All three experiments were run with the same $\lambda_{1,2,3} = [0.1, 0.4, 0.9]$ hyperparameters which control the loss function's dynamics. Comparisons for the Fluid Flow On Attractor 6, Pendulum 8, and Fluid Flow Box 10 experimental loss curves are shown below. We found that for all three experiments the Lusch network was unable to converge within the 500 epoch time frame and performed significantly worse in comparison to our two methods. In I, it is clear that the 2-Net Static Koopman method is the fastest in comparison to the 3-Net Hot Start, due to the fact that both require the roughly 9-minute Koopman



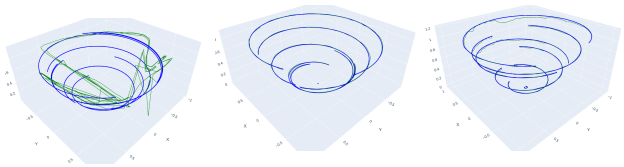
(a) Training Trajectory (b) Test Trajectory

Fig. 3: **Pendulum**, Ground-Truth trajectories show blue. Predicted trajectories show red.



(a) Training Trajectory (b) Test Trajectory

Fig. 4: **Fluid Flow Box**, Ground-Truth trajectories show blue. Predicted trajectories show red.



(a) Lusch Net (b) Three Net Hot Start (c) Two Net Static Koopman

Fig. 5: **Fluid Flow On Attractor**: The above graphs display the ground truth (blue) and estimated trajectories (green) sampled from the test set.

generation prior to training. In comparison to the recreation of Lusch's method, both of these take significantly longer, but perform much more accurately.

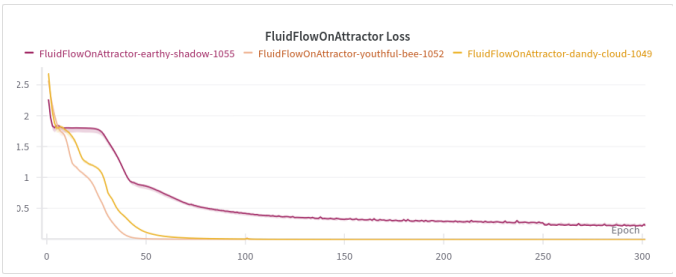
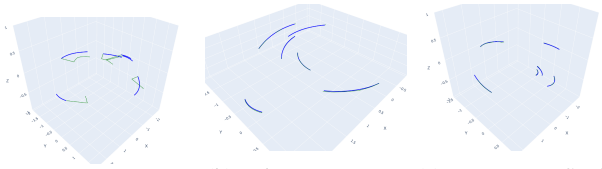


Fig. 6: **From Left to Right:** a) Lusch Net b) Three Net Hot Start c) Two Net Static Koopman.

Model	Train Acc	Val Acc	Test Acc	Avg Time to Convg (min)
Sznaier SDP	-	-	-	18.83
Lusch Net	27350.97	29495.405	30971.66	1.88
3-Net Hot	265.331	313.045	305.91	1.76
2-Net Static	326.710	336.095	357.082	1.44

TABLE I: Sznaier's results are omitted due to the model's inability to handle such large datasets. Accuracies are the summed MSELoss over each dataset between the ground-truth trajectories and their 1-step predictions for FluidFlowBox.



(a) Lusch Net (b) Three Net Hot Start (c) Two Net Static Koopman

Fig. 7: **Fluid Flow On Attractor**: The above graphs display the ground truth (blue) and estimated trajectories (green) sampled from the test set.

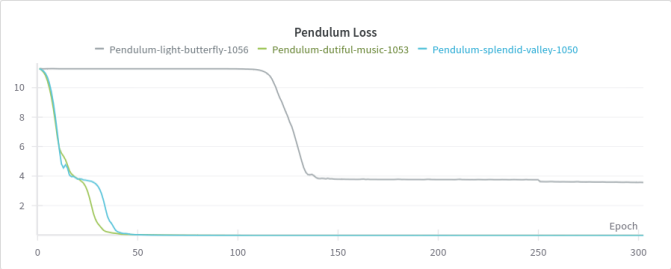
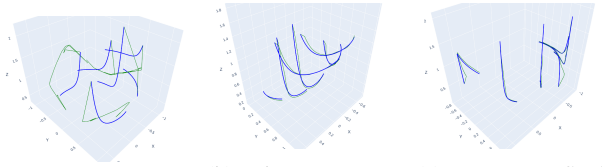


Fig. 8: **From Left to Right:** a) Lusch Net b) Three Net Hot Start c) Two Net Static Koopman.



(a) Lusch Net (b) Three Net Hot Start (c) Two Net Static Koopman

Fig. 9: **Fluid Flow On Attractor**: The above graphs display the ground truth (blue) and estimated trajectories (green) sampled from the test set.

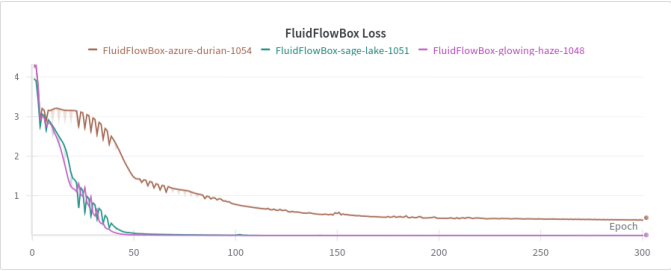


Fig. 10: **From Left to Right:** a) Lusch Net b) Three Net Hot Start c) Two Net Static Koopman.

V. Limitations

This work faced two notable limitations. First, due to time constraints, we were unable to refactor Sznaier's existing codebase to solve the semi-definite program (SDP) using multiple trajectories. Consequently, our implementation relied on single-trajectory data, potentially limiting the robustness and generalizability of the derived Koopman operator. Second, due to version incompatibilities, we were unable to fully replicate Lusch's three-network system which limited its efficacy in environments with continuous spectra. Despite these challenges, our hybrid framework demonstrates promising results, which we believe can be further proven in the future with more comparable baselines.

VI. Conclusion

In this work, we proposed a hybrid framework that combines the strengths of programmatic and learning-based approaches to identify linear representations for non-linear dynamics. By leveraging a two-stage optimization process, our method first uses a semi-definite program (SDP) to derive an optimized Koopman operator and then trains an autoencoder to approximate the coordinate transforms. This hybrid approach avoids the need for a priori specification of the operator's dimensionality while maintaining computational efficiency.

Our results demonstrate that the 3-network Hot Start method, which integrates the SDP-derived Koopman as a warm start for Lusch's auxiliary network, achieves the best performance. The 2-network Static Koopman method, which directly applies the SDP-derived Koopman operator without further optimization, also performed well, surpassing Lusch's original method and significantly outperforming Sznaier's SDP approach, which was unable to scale to larger datasets.These findings highlight the advantages of our hybrid framework in terms of accuracy, robustness, and scalability.

VII. Future Work

In future work, we intend to remove the limitations faced thus far with our baselines, to give a more fair comparison between the different methods. We also intend to improve integration of the two methods, such as replacing the SDP by trying to encode kernel information into the latent space directly.

References

[1] Petar Bevanda, Stefan Sosnowski, and Sandra Hirche. "Koopman operator dynamical models: Learning, analysis and control". In: *Annual Reviews in Control* 52 (2021), pp. 197–212. ISSN: 1367-5788. DOI: https://doi.org/10.1016/j.arcontrol.2021.09.002. URL: https://www.sciencedirect.com/science/article/pii/S1367578821000729.

[2] Steven L. Brunton et al. "Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control". In: *PLOS ONE* 11.2 (Feb. 2016). Ed. by Hans A Kestler, e0150171. ISSN: 1932-6203. DOI: 10.1371/journal.pone.0150171. URL: http://dx.doi.org/10.1371/journal.pone.0150171.

[3] Antonio Ionita. “Lagrange rational interpolation and its applications to approximation of large-scale dynamical systems”. Dissertation. Rice University, 2013. URL: https://hdl.handle.net/1911/77180.

[4] B. O. Koopman. “Hamiltonian Systems and Transformation in Hilbert Space”. In: *Proceedings of the National Academy of Sciences* 17.5 (1931), pp. 315–318. DOI: 10.1073/pnas.17.5.315. eprint: https://www.pnas.org/doi/pdf/10.1073/pnas.17.5.315. URL: https://www.pnas.org/doi/abs/10.1073/pnas.17.5.315.

[5] Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. “Deep learning for universal linear embeddings of nonlinear dynamics”. In: *Nature Communications* 9.1 (Nov. 2018). ISSN: 2041-1723. DOI: 10.1038/s41467-018-07210-0. URL: http://dx.doi.org/10.1038/s41467-018-07210-0.

[6] B. R. Noack et al. “A hierarchy of low-dimensional models for the transient and post-transient cylinder wake”. In: *Journal of Fluid Mechanics* 497 (2003), pp. 335–363. URL: https://api.semanticscholar.org/CorpusID:55584094.

[7] Peter J Schmid. “Dynamic mode decomposition and its variants”. In: *Annual Review of Fluid Mechanics* 54.1 (2022), pp. 225–254.

[8] Mario Sznaier. *A Convex Optimization Approach to Learning Koopman Operators*. 2021. arXiv: 2102.03934 [eess.SY]. URL: https://arxiv.org/abs/2102.03934.

[9] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. “A Data–Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition”. In: *Journal of Nonlinear Science* 25.6 (June 2015), pp. 1307–1346. ISSN: 1432-1467. DOI: 10.1007/s00332-015-9258-5. URL: http://dx.doi.org/10.1007/s00332-015-9258-5.